Transformers: https://hackage.haskell.org/package/transformers

State

```
-- Control.Monad.Trans.State
newtype StateT s m a = StateT { runStateT :: s -> m (a,s) }
type State s = StateT s Identity

instance MonadTrans (StateT s)
instance Monad m => Monad (StateT s m)
instance MonadFail m => MonadFail (StateT s m)
instance MonadPlus m => MonadPlus (StateT s m)

get :: Monad m => StateT s m s
put :: Monad m => s -> StateT s m ()
state :: Monad m => (s -> (a,s)) -> StateT s m a

get :: Monad m => StateT s m s
put :: Monad m => s -> StateT s m ()
gets :: Monad m => (s -> a) -> StateT s m a
modify :: Monad m => (s -> s) -> StateT s m ()
modify' :: Monad m => (s -> s) -> StateT s m ()

evalStateT :: Monad m => StateT s m a -> s -> m a
execStateT :: Monad m => StateT s m a -> s -> m s
withStateT :: (s -> s) -> StateT s m a -> StateT s m a
mapStateT :: (m (a,s) -> n (b,s)) -> StateT s m a -> StateT s n b
```

Reader

```
-- Controls.Monads.Trans.Reader
newtype ReaderT r m a = ReaderT { runReaderT :: r -> m a }
type Reader r m = ReaderT r m Identity

instance Monad m => MonadTrans(ReaderT r m)
instance Monad m => Monad(ReaderT r m)
instance (Functor m) => Functor (ReaderT r m)
instance (Monoid w, Functor m, Monad m) => Applicative (ReaderT r
m)
instance MonadFail m => MonadFail(ReaderT r m)
instance MonadPlus m => MonadPlus(ReaderT r m)
instance MonadFix m => MonadFix(ReaderT r m)

ask :: Monad m => ReaderT r m a
asks :: Monad m => (r -> a) -> ReaderT r m a
local :: (r -> r) -> ReaderT r m a -> ReaderT r m a

reader :: Monad m => (r -> a) -> ReaderT r m a
runReaderT :: Monad m => (r -> m a)
mapReaderT :: Monad m => (m a -> n b) -> ReaderT r m a -> ReaderT r
```

```
n b
withReaderT :: ('r -> r) -> ReaderT r m a -> ReaderT r' m a
```

Writer

```
-- Controls.Monads.Trans.Writer
newtype WriterT w m a = Writer { runWriter :: m (a,w) }
type Writer w a = WriterT w Identity a

instance (Monoid w) => MonadTrans (Writer w)
instance (Functor m) => Functor (Writer w m)
instance (Monoid w, Functor m, Monad m) => Applicative (WriterT w
m)
instance (Monoid w, Monad m) => Monad(WriterT w m)
instance (Monoid w, MonadFail m) => MonadFail(WriterT w m)
instance (Monoid w, MonadPlus m) => MonadPlus(WriterT w m)
instance (Monoid w, MonadFix m) => MonadFix(WriterT w m)

writer :: Monad m => (a, w) -> WriterT w m a
runWriterT :: Monad m => m (a, w)
execWriterT :: Monad m => WriterT w m a -> m w
mapWriterT :: Monad m => (m (a, w) -> n (b, w')) -> WriterT w m a
-> WriterT w' n b

tell :: Monad m => w -> WriterT w m ()
listen :: Monad m => WriterT w m a -> WriterT w m (a, w)
listens :: Monad m => (w -> b) -> WriterT w m a -> WriterT w m (a,
b)
pass :: Monad m => WriterT w m (a, w -> w) -> WriterT w m a
censor :: Monad m => (w -> w) -> WriterT w m a -> WriterT w m a
```

RWS

```
-- Controls.Monad.Trans.RWS
newtype RWST r w s m a = RWST { runRWST :: r -> s -> m (a, s, w) }
type RWS r w s = RWST r w s Identity

instance (Monoid w) => MonadTrans (RWST r w s)
instance (Functor m) => Functor (RWST r w s m)
instance (Monoid w, Functor m, Monad m) => Applicative (RWST r w s
m)
instance (Monoid w, Monad m) => Monad (RWST r w s m)
instance (Monoid w, MonadFail m) => MonadFail (RWST r w s m)
instance (Monoid w, MonadPlus m) => MonadPlus (RWST r w s m)
instance (Monoid w, MonadFix m) => MonadFix (RWST r w s m)

reader :: (Monoid w, Monad m) => (r -> a) -> RWST r w s m a
ask :: (Monoid w, Monad m) => RWST r w s m r
writer :: Monad m => (a,w) -> RWST r w s m a
tell :: Monad m => w -> RWST r w s m ()
...
```

State

State monad

State can be **simulated** by **explicit state** passing**.**
i.e. current state as **function input**
new state as **function output**
sequence of state actions as **function composition**
e.g.

```
compute :: Int -> Int -> State MyContext Int
compute x y = State (\s -> let                -- s = explicit
current state
  s' = ... use state ...
  in (x+y,s'))                                -- (x+y,s') = (action
result, explicit new state)
```

Problems:
1) manual state passing is tedious + **error prone**
2) state functions become **deep nested compositions**

Recognizing state functions as a **monad**, the **do expression** can be exploited.
i.e. **hide** explicit state
i.e. state functions expressed **sequentially** as opposed to nested composition.

e.g. do vs explicit state
With do expressions

```
compute :: Int -> Int -> State MyContext Int
compute x y = do
  s <- get                          -- get :: State MyContext
MyContext
  let s' = ... use state ...
  put s'                            -- put :: MyContext -> State
MyContext ()
  return x+y

-- Compose
task1 :: State MyContext Int
task1 = do
  y1 <- compute 1 2
  y2 <- compute 2 y1
  compute y1 y2

task2 :: State MyContext Int
task2 = do
  task1
  task1

-- Execute
(y', s') = runState task2 s0        -- s0=initial state s'=final
state y'=final result
```

Under the hood with explicit state passing

```
compute :: Int -> Int -> State MyContext Int
compute x y = state (\s -> let
  s' = ... use state ...
  return (x+y, s')

-- Compose
task1 :: MyContext -> (Int, MyContext)
task1 = state (\s - let                          -- deep nesting
  (y1, s2) = let
    (y1, s1) = compute 1 2 s
    in compute 2 y1 s1
  in compute y1 y2 s2

task2 :: MyContext -> (Int, MyContext)
task1 = state (\s -> let                         -- deep nesting
  (_, s1) = task1 s
  in task1 s1

-- Execute
(y', s') = task2 s0                              -- s0=initial state
s'=final state y'=final result
```

Reader monad
Writer monad
WRS monad
All forms of **state**.

	Reader	**Writer**	**State**	**RWS**
Description	Read only state e.g. global variable	Write only state e.g. logger	Read/write state	All combined
	Given state r, action m with result a r -> m a	Action m with current state w + result a m (a,w)	Given state s, action m with new state s + result a s -> m (a,s)	All combined r -> s -> m (a, s ,w)
Definition	`newtype ReaderT r m a = ReaderT { runReader:: r -> m a }`	`newtype WriterT w m a = WriterT { runWriter:: m (a,w) }`	`newtype StateT s m a = StateT { runState:: s -> m (a,s) }`	`newtype RWST r w s m a = RWST { runRWST :: r -> s -> m (a, s, w) }`

e.g.	rtask :: Reader [Int] Int rtask = do list <- ask -- read [1..10] v1 <- asks head -- read 1 return 100 -- 100 result = runReader rtask [1..10]	wtask :: Writer String Int wtask = do tell "Hello" -- write " tell "World" -- write return 100 -- result=100 -- ("HelloWorld", 100) (state, result) = runWriter wtask	rwtask :: State String Int rwtask = do message <- get -- read "World" put ("Hello" ++ message) -- write return 100 -- (result, state) = (100, "HelloWorld") (result,state) = runState rwtask "World"	

Strict vs Lazy version
[SO] Strict vs lazy: https://stackoverflow.com/questions/13186512/difference-between-haskells-lazy-and-strict-monads-or-transformers

The main difference is that a Strict transformer is **strict** in the **computation sequence**.
i.e. in the underlying monad bind

x >>= k

strict transformers use a **strict map k**, whereas lazy transformers use a **lazy map k**.
e.g.

```
listen :: Writer w m a -> Writer w m (a, w)
listen m = WriterT $ do
  (a, w) <- runWriterT m              -- Strict Writer: strict k = \(a, w) -> return ((a, w), w)
  return ((a, w), w)

  ~(a, w) <- runWriterT m             -- Lazy Writer: lazy k = \~(a, w) -> return ((a, w), w)
  return ((a, w), w)
```

Use lazy for **infinite** or large operations.

e.g.

```
import qualified Control.Monad.Trans.Writer.Lazy as L
```

```
import qualified Control.Monad.Trans.Writer.Strict as S

x = x :: (Int, String)

tellRepeat :: String -> L.Writer w m ()
tellRepeat s = L.tell s >> tellRepeat s        -- infinite tell
calls

tellRepeat' :: String -> S.Writer w m ()
tellRepeat' s = S.tell s >> tellRepeat s

take 10 $ L.execWriter $ tellRepeat "abc"      -- "abcabcabca"
take 10 $ S.execWriter $ tellRepeat' "abc"     -- BAD. Never
terminates
```

e.g. fixed point

```
(a, w) = L.runWriter $ L.tell a >> pure "abc"      -- OK. ("abc",
"abc")
(a, w) = S.runWriter $ S.tell a >> pure "abc"      -- BAD. Never
terminates

-- same with mfix
-- Works for both
L.runWriter $ mfix $ \a -> L.tell a >> pure "abc"  -- OK. ("abc",
"abc")
S.runWriter $ mfix $ \a -> S.tell a >> pure "abc"  -- OK. ("abc",
"abc")
```

Basics

```
---- Basics
--
--
task :: StateT Int Maybe String              -- state=Int
result=String
task :: StateT Int Maybe String              -- state=Int
result=String
task = StateT (\s -> Just (show s, s * 2))
task = do                                    -- same with do
  s <- get
  put (s*2)
  return (show s)
```

```
runState task 1        -- Just ("1",2)
runState task 2        -- Just ("2",4)
```

```
-- e.g. chain tasks
task1 :: Num s => StateT s Maybe s
```

```
task2 :: Num s => StateT s Maybe ()
task3 :: (Show s, Num s) => StateT s Maybe String
task4 :: (Show s, Num s) => StateT s Maybe String
task1 = gets (*2)
task2 = modify (*3)
-- combine
task3 = do
  v <- task1                          -- e.g. v = 4 if state = 2
  put (v+1)                           -- state=5
  task2                               -- state=15
  task2                               -- state=45. Apply multiple
times.
  gets show                           -- result="45" state=45
task4 = do
  v1 <- task3                         -- v1="45" state=45
  v2 <- task3                         -- v2="819" state=819
  task2                               -- state=819*3=2457
  return (v1 ++ "," ++ v2)
runStateT task4 2                     -- Just ("45,819",2457)


-- [MonadTrans] lifting monad to StateT
--
lift :: Monad m => m a -> StateT s m a
return 1            -- StateT (\s -> [(1,s)]
lift [1..10]        -- StateT (\s -> [(1,s),(2,s),...]
lift (Just 1)       -- StateT (\s -> Just (1,s))
lift Nothing        -- StateT (\s -> Nothing)
```

```
e.g. equivalent lift if Traversable

liftT :: Traversable m => m a -> StateT s m a
liftT = StateT . traverse (,)
```

```
-- applicative
mf = (*) [1..10]               -- [(1*),(2*),...]
lift mf                        -- StateT (\s -> [(1*,s),(2*,s)...])
lift mf <*> get                -- StateT (\s -> [(1*s,s),
(2*s,s)...])
runState (lift mf<*>get) 1     -- [(1,1),(2,1),...,(10,1)]
```

State utils

```
---- Utils
--
-- state read/write
get :: Monad m => StateT s m s
put :: Monad m => s -> StateT s m ()
gets :: Monad m => (s -> a) -> StateT s m a
modify :: Monad m => (s -> s) -> StateT s m ()
modify' :: Monad m => (s -> s) -> StateT s m ()
get = state (\s -> (s,s))
```

```
put s = state (\_ -> ((), s))
gets f = get >>= return.f
modify f = get >>= put.f
```

```
Analogous to React hooks

// React Hook
const [value, setValue] = useState();
setValue(value + 1);
return <div>value=${value}</div>

--  Haskell state monad
do
 value <- get            -- get :: m (s,s)
 put (value + 1)         -- put :: s -> m ((),s)
 return "value" ++ show value
```

```
-- Basic
do
  s <- get                  -- read state
  s <- gets f               -- read state + map i.e. s=f(state)
  _s <- get; let s=f _s     -- same
  put 10                    -- write state
  get >>= put               -- write state with result
  gets (+2) >>= put
  s <- put 10 >> get        -- write state and read
  modify f                  -- modify state directly i.e. new
state=f(state)
  s <- get; put (f s)       -- same
```

```
-- e.g.
task = do
  s <- get                -- s="Hello"
  s' <- gets reverse      -- s'="olleH"
  put (s ++ "World")      -- "HelloWorld"
  modify reverse          -- "dlroWolleH"
  return 0
runState task "Hello"     -- (0, "dlroWolleH")
```

```
-- state execution
evalStateT :: Monad m => StateT s m a -> s -> m a
execStateT :: Monad m => StateT s m a -> s -> m s
evalStateT task = (fst<$>).(runState task)
execStateT task = (snd<$>).(runState task)

-- e.g.
task = return 1 :: StateT a Maybe Int
runStateT task True     -- Just (1,True)
evalStateT task True    -- Just 1
```

```
execStateT task True    -- Just True

-- state transform
withStateT :: (s -> s) -> StateT s m a -> StateT s m a
  -- map state only
mapStateT :: (m (a,s) -> n (b,s)) -> StateT s m a -> StateT s n b
   -- map whole
withStateT f task
modify f >> task        -- same
```

***) GOTCHA: Control.Monad.Trans.Writer.Strict vs Control.Monad.Trans.Writer.CPS**
Writerを使ってはならない: https://qiita.com/ruicc/items/28b1cd2df215ce724e26

[Infinite negativity] WriterT space leaks: https://journal.infinitenegativeutility.com/writer-monads-and-space-leaks
mail (Gonzalez)
  part1: https://mail.haskell.org/pipermail/libraries/2012-October/018599.html
  part2: https://mail.haskell.org/pipermail/libraries/2013-March/019528.html

Problem
WriterT is NOT really strict:
1) **strictness**: Monoid w is NOT strict i.e. thunks of w not **collapsed**
2) **blocking**: monad returns nothing until everything is evaluated

Solution
Use CPS

BAD: Strict Writer

```
instance (Monoid w, Monad m) => Monad (WriterT w m) where
    m >>= k  = WriterT $ do
        (a, w)  <- runWriterT m
        (b, w') <- runWriterT (k a)
        return (b, w `mappend` w')         -- BAD 2: blocking i.e.
does not return until both m and k finish
                                           -- BAD 1: w `mappend` w'
is not evaluated to WHNF
```

e.g.

```
x :: WriterT w m Int
x = do
   a <- m
   b <- f a
   c <- g b
   d <- h c
   return (a + d)
```

Expands to

```
do
  (a, w0) <- runWriter m                   --|  all this kept in
memory
```

```
  (r, w1) <- do                                    |
     (b, w1') <- runWriter (f a)                   |
     (s, w2)  <- do                                |
        (c, w2') <- runWriter (g b)                |
        (t, w3)  <- runWriter (h c')               |
        return (t, w2' `append` w3)                |
                                                   |
     return (s, w1' `append` w2)                   |--  w1 kept in memory
until here
                                                   |
  return (r, w0 `mappend` w1)                   --|--  w0 kept in memory
until here
```

Good: CPS Writer

```
writer :: (Monoid w, Monad m) => (a, w) -> WriterT w m a
writer (a, w') = WriterT $ \ w ->
    let wt = w `mappend` w' in wt `seq` return (a, wt)      -- GOOD
1: strict evaluation to WHNF

instance (Monad m) => Monad (WriterT w m) where
    m >>= k = WriterT $ \ w -> do
        (a, w') <- unWriterT m w
        unWriterT (k a) w'                 -- GOOD 2: non-blocking
```

==========

OLD

```
---- Reader
reader :: Monad m => (r -> a) -> ReaderT r m a
runReaderT :: Monad m => (r -> m a)
mapReaderT :: Monad m => (m a -> n b) -> ReaderT r m a -> ReaderT r
n b
withReaderT :: ('r -> r) -> ReaderT r m a -> ReaderT r' m a

ask :: Monad m => ReaderT r m a
asks :: Monad m => (r -> a) -> ReaderT r m a
local :: (r -> r) -> ReaderT r m a -> readerT r m a


---- Writer
writer :: Monad m => (a, w) -> WriterT w m a
runWriterT :: Monad m => m (a, w)
execWriterT :: Monad m => WriterT w m a -> m w
mapWriterT :: Monad m => (m (a, w) -> n (b, w')) -> WriterT w m a
-> WriterT w' n b
```

```
tell :: Monad m => w -> WriterT w m ()
listen :: Monad m => WriterT w m a -> WriterT w m (a, w)
listens :: Monad m => (w -> b) -> WriterT w m a -> WriterT w m (a,
b)
pass :: Monad m => WriterT w m (a, w -> w) -> WriterT w m a
censor :: Monad m => (w -> w) -> WriterT w m a -> WriterT w m a
-- write state
tell "hello"                  -- state=((), "hello")

-- read state
listen (tell "hello")         -- state=("hello","hello")

-- map state
censor reverse (tell "hello")                 -- state=((),
"olleh")
pass $ tell "hello" >> return ((), reverse)    -- same with pass



----  RWST
reader :: (Monoid w, Monad m) => (r -> a) -> RWST r w s m a
ask :: (Monoid w, Monad m) => RWST r w s m r
...

writer :: Monad m => (a,w) -> RWST r w s m a
tell :: Monad m => w -> RWST r w s m ()
...

-- [MonadPlus] <|> : join StateT
--
-- i.e. algebraic sum of StateT
dup3 :: MonadPlus m => State a m a
dup3 = get <|> get <|> get                          -- duplicate 3
times
compute3 :: (Num a, MonadPlus m) => State a m a
compute3 = get <|> gets(*2) <|> gets (*3)

-- e.g.
runState dup3 0 :: Maybe (Int,Int)      -- Just (0,0)
runState dup3 0 :: [(Int,Int)]          -- [(0,0),(0,0),(0,0)]

runState compute3 1 :: Maybe (Int,Int)  -- Just (1,1)
runState compute3 1 :: [(Int,Int)]      -- [(1,1),(2,1),(3,1)]


-- [MonadPlus] mzero: short circuit StateT
-- i.e exit early if error
--
assertEven :: MonadPlus m => State Int m ()
assertEven = do
  s <- get
  unless (even s) mzero
-- use
```

```
task = do
  assertEven
  gets show
```

```
runState task 0         -- Just ("0",0)
runState task 1         -- Nothing
```

```
// e.g. JavaScript
// doSomething (x:number,y:number) => string
function doSomething(x,y) {
 ... use state ...
}

-- as State
doSomething :: Int -> Int -> State (IO ()) String      -- state=(IO
(), [Int]) result=String
```

e.g. similar syntax as JavaScript **React** framework **Hooks**

```
// React Hook
const [value, setValue] = useState();
setValue(value + 1);
return <div>value=${value}</div>


--  Haskell state monad
do
 value <- get            -- get :: m (s,s)
 put (value + 1)         -- put :: s -> m ((),s)
 return "value" ++ show value
```

e.g. State monad as syntactic sugar

	JavaScript	**Haskell without State monad** i.e. explicit state passing	**Haskell with State monad**
Signature	`compute (x:number, y:number) => number`	`compute :: Int -> Int -> (MyState -> (Int, MyState))`	`compute :: Int -> Int -> State MyState Int`

Function	function compute(x,y) {
	
	  ... use state ...
	  return x + y;
	}
		compute state x y = (\s -> let
		  s' = ... use state ...
		  in (s', x + y))
			compute x y = do
			  s <- get
			  let s' = ... use state ...
			  put s'
			  return (x + y)

Compute	const z1 = compute(0,1);
	const z2 = compute(1,z1);
	const z3 = compute(z1,z2);
	return [z1,z2,z3]
		let
		  (state1, z1) = compute state0 0 1
		  (state2, z2) = compute state1 1 z1
		  (state3, z3) = compute state2 z1 z2
		  in (state3, [z1,z2,z3])
			do
			  z1 <- compute 0 1
			
			  z2 <- compute 1 z1
			  z3 <- compute z1 z2
			  return [z1,z2,z3]
			
			-- can chain more
			task1 :: State MyState Int
			task1 = do
			  z1 <- compute 0 1
			
			  z2 <- compute 1 z1
			  ....
			task2 :: State MyState Int
			task2 = do
			  task1
			  task1
			  ...

e.g. a stateful operation including writing to console.

```
// JavaScript
const items = [];
function compute(x) {
  console.log(x);
  items.push(x);
  return 2 * x;
}
```

Haskell with explicit state passing

```
type ComputeStateData = ([Int], IO ())

compute :: (Int, ComputeStateData) -> (Int, ComputeStateData)
compute (x, (items, console)) =
  let items' = items ++ [x]
      io' = do console; putStrLn $ show x        -- print to IO
      in (x * 2, (items', io'))                      --

-- compute
let
  state0 = ([], pure())
  (y1,state1) = compute state0 1              -- items=[1] y1=2
  (y2,state2) = compute (y1,state1)           -- items=[1,2] y2=4
  (y3,state3) = compute (y2,state2)           -- items=[1,2,4] y3=8

-- print result
let
 (items,console) = state3
do
  console
  putStrLn $ show items ++ show y3
```

Haskell with state monad

```
type ComputeStateData = ([Int], IO ())

compute :: Int -> State ComputeStateData Int
compute x = do
  (items, console) <- get                      -- use state
  let items' = items ++ [x]
      console' = do console; putStrLn $ show x
  put (items', console')                       -- update state
  return (x * 2)

-- compute
let
  task1 :: State ComputeStateData Int
  task1 = do
    y1 <- compute 1                           -- items=[1] y1=2
    y2 <- compute y1                          -- items=[1,2] y2=4
    compute y2                                -- items=[1,2,4] y3=8

-- print result
let
  (y3, (items, console)) =
     runState computeAll ([], pure ())        -- initial state
 in do
  console
  putStrLn $ show items ++ show y3
```

```
-- chain more
let
  task2 :: State ComputeStateData Int
  task2 = do
    task1
    task1
    task1
```